

1.Introduction

Brain tumours are one of the most serious intracranial conditions, and magnetic resonance imaging is the modality used to detect them, plan their treatment and follow them over time, it is non-ionising, it gives strong soft-tissue contrast and it is available in most hospitals. Segmentation is used to outline the exact boundary of the tumour on the slice, separating the lesion from the healthy brain around it, so that the volume can be measured, compared against a previous study and used to plan a radiotherapy dose. This is harder on brain MRI than it looks, gliomas infiltrate the tissue around them so the margin is genuinely ambiguous even between two expert raters, intensity is not uniform across the field of view, and the scans arrive at several different native resolutions.

The bottleneck is the manual outlining itself. In current practice a radiologist or a dosimetrist draws the lesion by hand on every slice of the study, which takes roughly 10 to 20 minutes per study, and more importantly it is inconsistent, the same observer will not draw the same boundary twice and two observers will differ more still. Because response assessment works by comparing a volume today against a volume from three months ago, that inconsistency feeds directly into the treatment decision. A semi-automated or automated step does not replace the clinician here, it removes the drawing work and leaves the judgement where it belongs, which also helps where expert coverage is limited.

Dataset: BRISC 2025, brain tumour MRI, accessed through <https://www.kaggle.com/datasets/briscdataset/brisc2025> and accompanied by Fateh et al. (2025). The release provides 6,000 T1-weighted slices with radiologist-reviewed pixel-level masks across glioma, meningioma, pituitary and no-tumour classes in axial, coronal and sagittal planes. The segmentation subset used here holds 3,933 training and 860 test image/mask pairs and all of them contain a tumour. The images arrive at several native resolutions (mostly 512×512 , with a tail down to about 200×200 and up to 1427×1275), so every image and mask is resampled to a common 512×512 before anything else happens and every metric in this chapter is computed in that 512×512 space, which is what makes the Task 1 and Task 2 numbers directly comparable to each other. Two tasks were addressed: Task 1 uses classical image processing with one user-drawn box and a supervised pixel classifier, Task 2 uses a deep learning model that is structurally pruned to cut its floating point operations and to bring it inside the module's two million parameter limit.

1.1 literature review

Interactive segmentation has a long lineage and the interaction style chosen here sits inside it. GrabCut (Rother et al., 2004) established the loose bounding box as the interaction, random walker segmentation (Grady, 2006) established seed-based propagation from a click or a scribble, and ITK-SNAP (Yushkevich et al., 2006) brought semi-automated active contour segmentation into routine neuroimaging use. More recent work such as DeepIGeoS (Wang et al., 2018) combines a learned model with user interaction for medical images specifically. The reason this family is still worth using in 2026 is not accuracy, it is accountability, a system that produces a contour with no human in the loop has to be validated to a much higher standard than one that assists a clinician who stays responsible for the result. On the deep learning side the U-Net (Ronneberger et al., 2015) is still the backbone of almost all medical segmentation work, and nnU-Net (Isensee et al., 2021) showed that a well-configured U-Net still matches or beats most architectural innovations across many datasets, which sets a deliberately high bar for any model calling itself novel and is the reason the benchmark in this chapter is a tuned U-Net rather than one left on default settings. Two lines

of improvement are relevant to the design in Section 3.1. The first is attention: Attention U-Net (Oktay et al., 2018) introduced gates on the skip connections that suppress the irrelevant regions, and transformer based models such as TransUNet (Chen et al., 2021) and Swin-UNet (Cao et al., 2022) add global self-attention to address the fact that a convolution can only ever integrate information inside its own receptive field, the BRISC dataset paper itself proposes a Swin-based hybrid (Fateh et al., 2025). The second is efficiency: structured channel pruning (Li et al., 2017) removes whole filters and therefore genuinely reduces FLOPs, unlike unstructured magnitude pruning which leaves the tensor shapes intact, and Liu et al. (2019) showed that the pruned architecture often matters more than the weights it inherits, which is what motivates the round-by-round fine-tuning used in Section 3.3.

Three of these directly shaped what was built. The gate placement pattern of Oktay et al. (2018), four gates sitting between the encoder and the decoder on the skip connections, is the starting point the dual attention gate of the innovation model extends (Section 3.1). The L1 filter ranking of Li et al. (2017) is the exact criterion implemented in the pruning stage (Section 3.3). nnU-Net (Isensee et al., 2021) is used later in Section 4.3 as the state of the art comparison target, and the interaction taxonomy above is what places Task 1's single box inside an established family rather than making it an arbitrary choice.

2. Task 1 Classical (semi-automated) Segmentation

2.1 Methodology

Hardware specifications: Apple M1 Max, 10 CPU cores, 32 GB unified memory, macOS.

Task 1 requires no GPU at any stage, neither for training nor for inference.

Software: Python 3.11.1, NumPy 2.4.6, SciPy 1.17.1, scikit-image 0.26.0, OpenCV (opencv-python) 5.0.0, scikit-learn 1.9.0, joblib 1.5.3. Random seed 1 for the training sample, evaluation seed 2 for the test draw and the simulated boxes. No TensorFlow, no Haar cascade and no template matching is used anywhere in Task 1, as the brief requires.

Data: 3,933 training and 860 held-out test pairs. The model is built from 400 randomly chosen training scans and the headline is reported on 50 unseen test scans as the question paper requires. The test scans were not used for tuning, for threshold selection or for any parameter choice, and every tuned value is a fixed named constant in task1/config.py that is set once and never re-tuned on the test set. All images are resampled to 512×512 and converted to single channel greyscale before the pipeline, and the only inclusion criterion is that the scan carries a labelled lesion.

The interaction is one loose rectangle dragged around the lesion by the user. For batch evaluation that box has to be simulated, so it is generated from the ground truth bounding box with each of the four sides pushed outwards by a random 2 to 20% of the lesion size, which means the box is never tight and never repeats between runs. Only the four box coordinates reach the algorithm, the mask itself is never seen at test time, this is the standard way interactive segmentation is evaluated and it is what stops the simulated box from leaking the answer.

The algorithm produces a binary mask from the input scan through four stages:

- ❖ The preprocessing stage applies CLAHE and z-score normalisation over the head region, then crops the user's box with 35% padding added around it and resamples that region of interest to 128×128 . The padding is there because the classifier has to see some healthy tissue to compare the lesion against, and the resample is a scale normalisation, a small pituitary tumour and a large glioma end up looking the same size to the classifier.

- ❖ The feature stage describes every pixel of the region with 41 features. 32 of them are appearance: raw intensity, bilateral filtered intensity, Gaussian smoothing at sigma 1, 2, 4, 8 and 16, the Sobel gradient magnitude at each of those scales, the Laplacian of Gaussian at each of them, the Hessian eigenvalue pair at sigma 1, 2, 4 and 8, the local mean and standard deviation over 5, 11 and 21 pixel windows and a 9×9 median. 4 of them are geometry: the elliptical distance from the centre of the box where 1.0 is the box edge, the normalised row and column offsets, and an inside/outside box flag. The last 5 are context: the difference to the box core mean, the difference to the outside-box mean, a z-score against the core, the core mean and the outside mean.
- ❖ The classification stage is a Random Forest with 120 trees, min_samples_leaf of 20, max_features set to sqrt and class_weight set to balanced. It is trained on 400 scans × 700 sampled pixels, which is 280,000 rows, drawn 50% lesion, 25% near the boundary and 25% far background, deliberately weighted towards the boundary because that is where the errors actually happen. Training takes 36 seconds on CPU.
- ❖ The post-processing stage smooths the probability map with a Gaussian at sigma 1.5, thresholds it at 0.5, applies a morphological closing, fills the holes, keeps the largest connected component and pastes the result back into the full 512×512 frame.

Shown in Figure 1 below is the same four stages drawn end to end, from the box the user drags to the mask that comes back.

Task 1 - the pipeline from the user's box to the mask

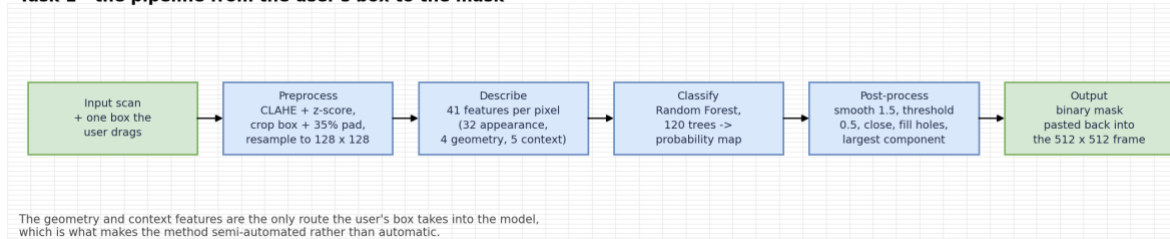


Figure 1 Task 1 pipeline, from the user's box to the binary mask

The geometry and the context groups are the only route the user's input takes into the model, and that is what makes this method semi-automated rather than automatic. The trained forest confirms how much work that input does, the box_elliptical_dist feature alone carries 26.6% of the total feature importance with inside_box, box_dist_x and box_dist_y also sitting in the top six, so the single most informative thing the classifier knows about a pixel is where it sits relative to the box the clinician drew.

Table 1 Task 1 pipeline parameters

Stage	Parameter	Value
Common	working frame all metrics are computed in	512 × 512
Preprocess	CLAHE and z-score normalisation	over the head region
Preprocess	padding added around the user box	35% of the box
Preprocess	region of interest resample size	128 × 128
Features	features per pixel	41 (32 appearance, 4 geometry, 5 context)
Features	Gaussian / LoG / Sobel scales (sigma)	1, 2, 4, 8, 16
Features	Hessian eigenvalue scales (sigma)	1, 2, 4, 8

Classifier	trees, min_samples_leaf	120, 20
Classifier	max_features, class_weight	sqrt, balanced
Training	scans, pixels per scan, total rows	400, 700, 280,000
Training	pixel sampling balance	50% lesion, 25% boundary, 25% background
Post-process	probability smoothing sigma, threshold	1.5, 0.5
Evaluation	simulated box slack per side	random 2–20%

Flowchart for the segment() routine in task1/segmenter.py, with the two points where it refuses to invent an answer (shown Figure 2):

task1/segmenter.py - segment(image, box): four stages

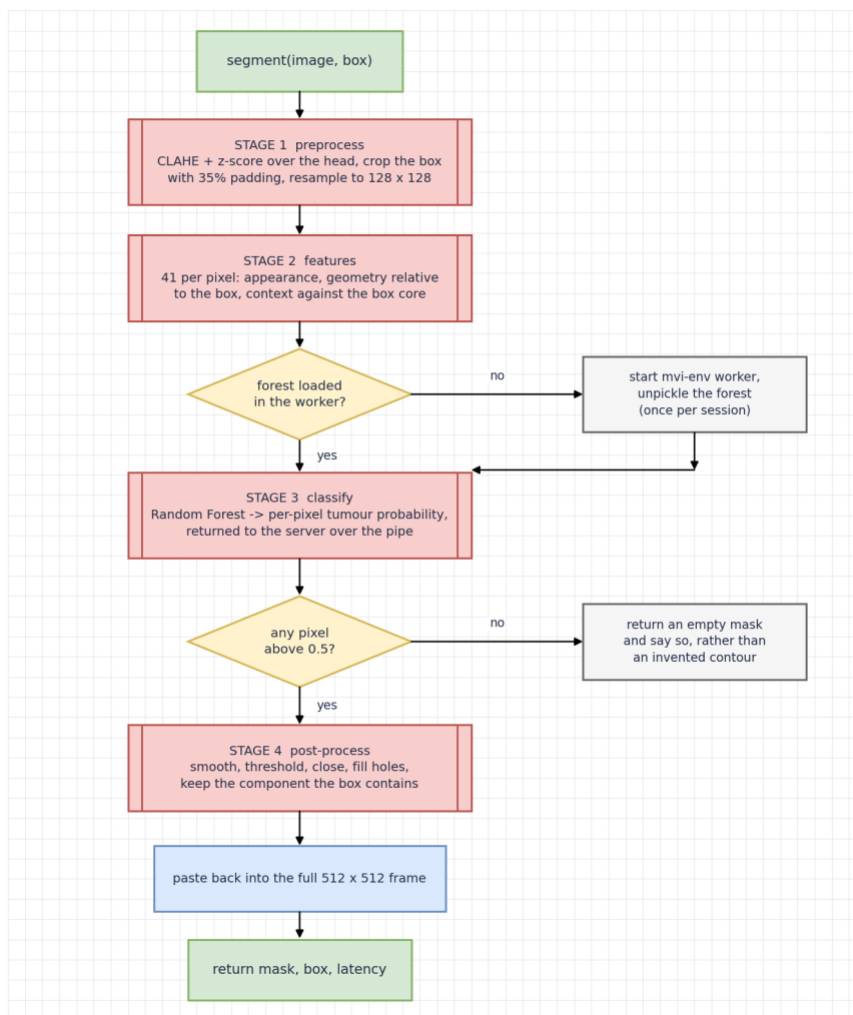


Figure 2 task1/segmenter.py segment() flowchart, four stages and two decisions

The unsupervised comparison methods are Otsu thresholding (Otsu, 1979), k-means clustering on intensity with k set to 3, classical seeded region growing, and morphological Chan-Vese active contours (Marquez-Neila et al., 2014). All four receive the identical simulated user box and the identical preprocessing, the only thing that changes between them is the segmentation decision itself, so the comparison is on the decision and not on the pipeline around it.

2.2 Results

Table 2 shows the proposed method against the four unsupervised baselines on the same 50 unseen scans, having a higher Dice, IoU and accuracy with a lower HD95 (in pixels) is better when achieved.

The four metrics are defined as follows. The Dice similarity coefficient is $2|A \cap B| \div (|A| + |B|)$ where A is the predicted lesion mask and B is the ground truth mask, IoU is $|A \cap B| \div |A \cup B|$, pixel accuracy is the proportion of correctly classified pixels and is defined by $(TP + TN) \div (TP + TN + FP + FN)$, and HD95 is the 95th percentile of the distance in pixels from each predicted boundary point to the nearest ground truth boundary pixel, the 95th percentile rather than the maximum so that one stray pixel does not dominate the number.

Table 2 Task 1 proposed and unsupervised baselines on the 50 image test run

Method (segmentation decision)	Dice	Median Dice	IoU	Accuracy	HD95 (px)	Sensitivity	Precision	s/image
Proposed (Random Forest + box)	0.8740	0.9284	0.8016	0.9930	11.37	0.9143	0.8635	0.063
Chan-Vese active contour	0.7813	0.8886	0.6910	0.9921	16.12	0.7382	0.8724	0.131
Otsu threshold	0.7744	0.8438	0.6714	0.9868	19.02	0.8333	0.7825	0.005
k-means (k = 3)	0.7230	0.7235	0.5991	0.9801	23.72	0.8345	0.7300	0.029
Seeded region growing	0.6594	0.7515	0.5423	0.9840	21.65	0.6288	0.8436	0.005

The proposed method beats the best unsupervised comparator, which is the morphological Chan-Vese contour, by 9.3 Dice points and it reduces the boundary error by 29% at the same time (HD95 11.37 against 16.12). 43 of the 50 scans score a Dice of 0.85 or better and 45 of them score 0.70 or better, so the distribution is not carried by a handful of easy cases, and the median of 0.9284 sitting well above the mean of 0.8740 says the remaining failures are a small number of bad cases rather than a general weakness. Accuracy is the least useful number in the table and is only reported because the brief asks for it, every method scores above 0.98 simply because the tumour occupies a small fraction of a 512×512 frame and calling everything background is already almost right.

Table 3 Task 1 Dice by tumour type (50 scans)

Tumour type	Dice	n
Meningioma	0.9430	18
Pituitary	0.8965	19
Glioma	0.7454	13

The split by tumour type is the more informative result. Meningiomas and pituitary adenomas are compact and well circumscribed so a box plus an intensity-and-texture classifier handles them easily, gliomas infiltrate the tissue around them and their margin is ambiguous even between expert human raters, which is the ceiling any reported glioma Dice should be read against rather than being treated as an implementation defect.

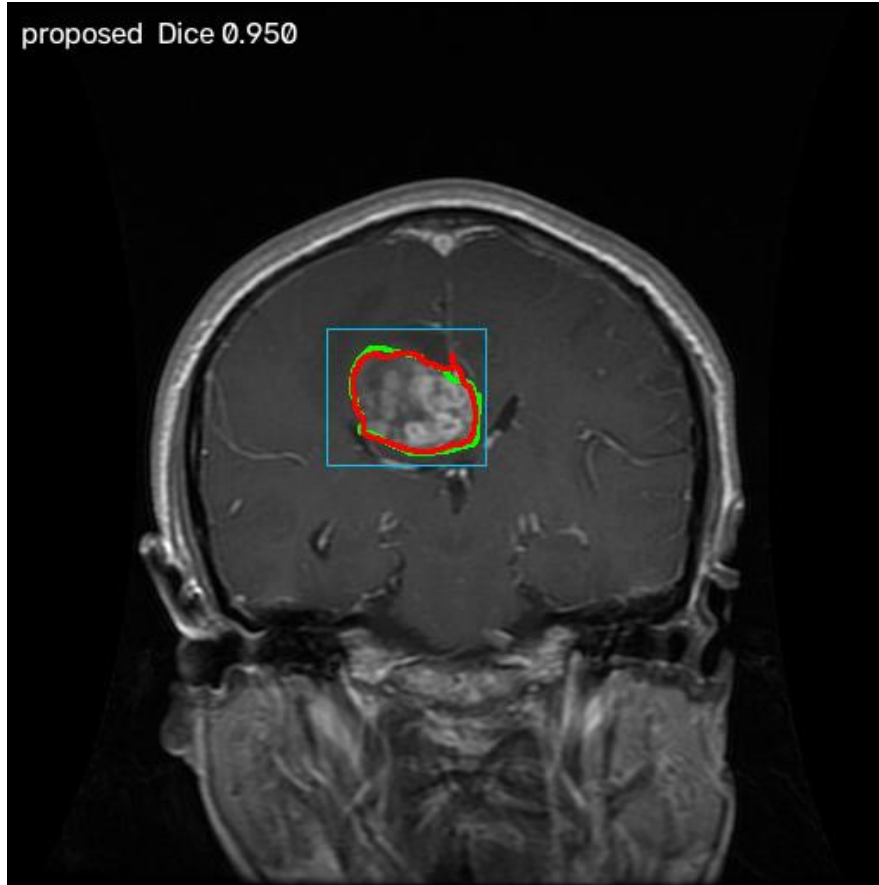


Figure 3 Task 1 output on a meningioma, white/cyan box: the simulated user box, green: ground truth, red: prediction

2.3 Model compression

The forest that is shipped is not the largest forest that was trained. Table 4 sweeps the number of trees against the minimum leaf size and reports the storage, the Dice and the latency for each, all measured on the same 50 scans.

Table 4 Task 1 Random Forest compression sweep

Trees	min_samples_leaf	Total nodes	Size (MB)	Dice	Latency (s)
300	4	4,977,358	92.48	0.8748	0.085
200	8	2,259,548	46.47	0.8748	0.077
150	12	1,304,474	28.86	0.8742	0.067
120	20	731,774	17.85	0.8740	0.062
100	30	449,548	11.79	0.8733	0.064
60	40	216,692	5.97	0.8727	0.049

The shipped configuration of 120 trees at a leaf size of 20 is 81% smaller and 28% faster than the largest forest for a Dice loss of 0.09%, which is well inside the variation caused by the random box slack alone. Worth noting that this saving is a real one in a way that magnitude pruning of a network is not, the nodes genuinely disappear from the file and from the traversal, so the storage and the latency both move together rather than only the storage moving.

3. Task 2 Deep Learning Segmentation

3.1 Methodology

Hardware specifications: Apple M1 Max, 10 CPU cores, 32 GB unified memory, with the integrated GPU used through tensorflow-metal, macOS. No datacentre GPU was used at any stage of this work, which is a deliberate constraint and is what most of the carbon argument in Section 4.6 rests on.

Software: Python 3.11.1, TensorFlow 2.13.0, tensorflow-metal 1.0.1, NumPy 1.24.3, SciPy 1.11.4, scikit-image 0.22.0, OpenCV 4.8.1.78, scikit-learn 1.3.2, pandas 2.0.3. Random seed 42 for training, evaluation seed 2. The networks train at 192×192 and their predictions are pushed back up to 512×512 before scoring, so Task 1 and Task 2 are measured on exactly the same masks in exactly the same space.

Two models were compared using the same images, the same preprocessing, the same resolution, the same metrics and the same evaluation code:

- ❖ Benchmark: a conventional U-Net with max pooling downsampling kept as the brief requires, with Particle Swarm Optimisation used to tune the layer widths, the dropout, the activation and the learning rate.
- ❖ Innovation (MHA-ResUNet): a U-Net with (1) residual blocks throughout, two Conv-BN-activation layers with the input added back through a 1×1 projection shortcut, (2) multi-head self-attention at the bottleneck with 4 heads and a fixed key dimension of 64, and (3) a dual attention gate on every skip connection, which is the novel component.

The dual gate is where this model departs from Attention U-Net rather than merely reimplementing it. A standard attention gate produces one spatial map that says where to attend but treats all 256 channels of a skip as equally relevant, this gate adds a squeeze-and-excitation branch alongside the spatial branch so the skip is reweighted by where and by which features at the same time, the skip output being the skip multiplied by the spatial attention and then by the channel attention. Both branches are driven by the decoder's gating signal, so the deeper semantic features are the ones deciding what the shallow features are allowed to pass. The self-attention at the bottleneck addresses a failure mode that is specific to brain MRI, a convolution cannot compare a candidate lesion against the corresponding structure on the other side of the midline because the two sit outside each other's receptive field, whereas at the 12×12 bottleneck grid every position attends to every other one. The key dimension is deliberately held at a constant 64 instead of being set to channels divided by heads, because if it scaled with the channel count then pruning the bottleneck would reshape the attention weights in a way that cannot be sliced and the pruned model would not be able to inherit them, which is a design decision taken in Section 3.1 purely to make Section 3.3 possible.

In Figure 4 the residual encoder and decoder are shown with the gates that sit between them, and every width is given as it was trained and then as it was pruned.

task2/models.py - build_mha_resunet(): encoder, bottleneck, decoder

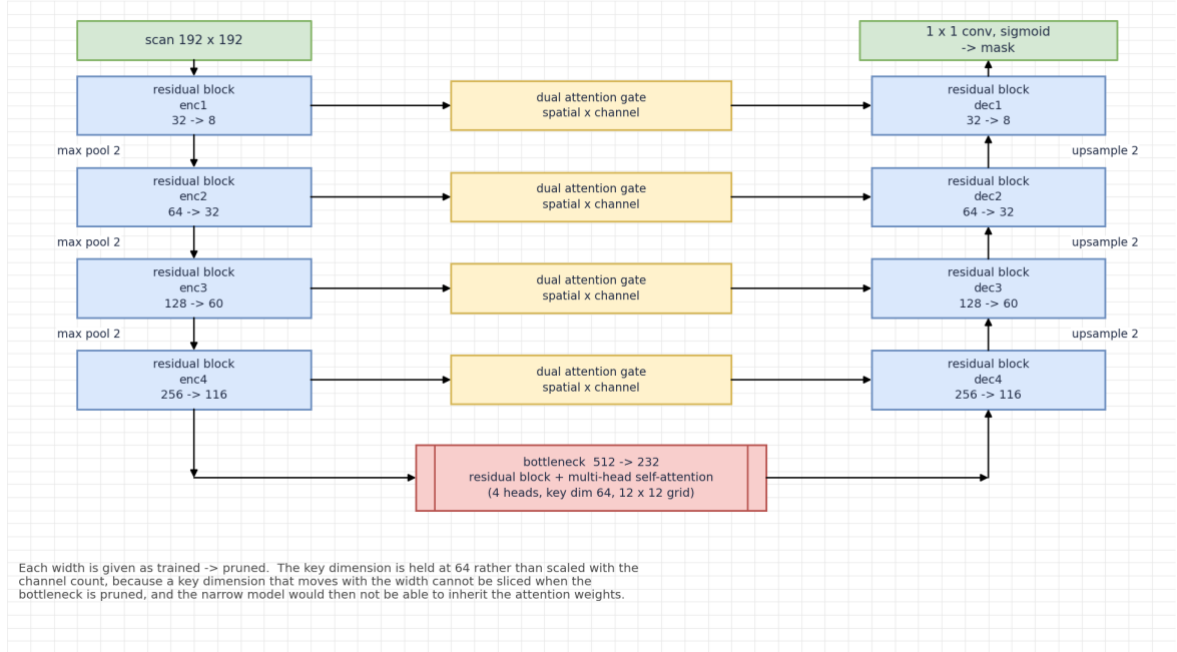


Figure 4 The MHA-ResUNet, with each width given as trained then pruned. The dual attention gates sit on the four skip connections and the self-attention sits at the bottleneck

Particle Swarm Optimisation was used on the benchmark with the standard velocity update, inertia 0.7 and both the cognitive and social coefficients at 1.5, following $v \leftarrow w \cdot v + c_1 \cdot r_1 \cdot (\text{personal_best} - x) + c_2 \cdot r_2 \cdot (\text{global_best} - x)$ and $x \leftarrow x + v$. The search used 6 particles over 4 iterations, which is 24 fitness evaluations, each one a 6 epoch run on 1,000 scans at 128x128, and the whole search took 30.4 minutes. Searching at a smaller resolution on a subset is what keeps the search affordable, the ranking of the candidates is what matters at this stage rather than the absolute Dice any of them reaches. The search itself is drawn in Figure 5.

task2/ps0.py - the search that tunes the benchmark U-Net

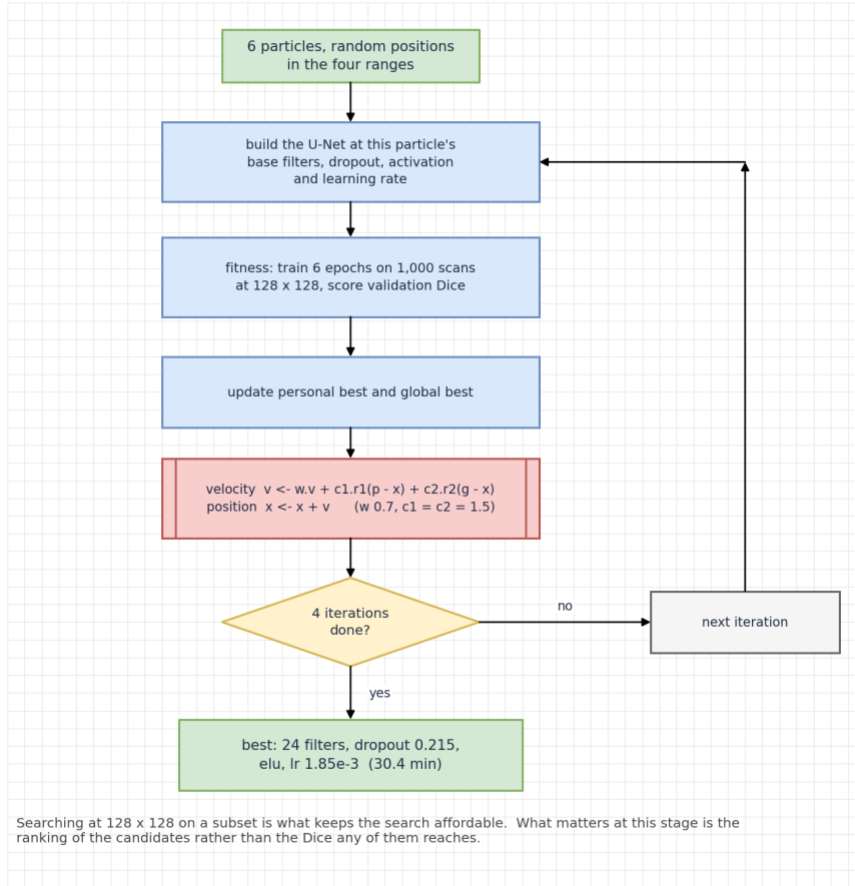


Figure 5 The PSO search that tunes the benchmark U-Net, 6 particles over 4 iterations

Table 5 Final tuned hyperparameters

Setting	Benchmark U-Net (PSO tuned)	Innovation MHA-ResUNet
Base filters	24 (searched over 8–48)	32, giving 32/64/128/256 and a 512 bottleneck, pruned to 8/32/60/116/232 for the submitted model
Dropout	0.215 (searched over 0.0–0.4)	0.1, fixed
Activation	elu (searched over relu/elu)	relu
Learning rate	1.85×10^{-3} (searched over 1×10^{-4} – 5×10^{-3} , log scale)	5×10^{-4}
Attention	none	4 heads, key dimension 64, dual gate on every skip
Input size, batch size	192×192 , 16	192×192 , 16
Epochs	40, early stopping patience 8	40, early stopping patience 8
Loss	BCE + Dice, equal weighting	BCE + Dice, equal weighting
BatchNorm momentum	0.9	0.9

Both models were then trained under identical final settings, Adam at the learning rate in Table 5, batch size 16, 40 epochs with early stopping on the validation Dice at a patience of 8

and the learning rate halved on plateau, an 80/20 train and validation split, and a combined binary cross-entropy and Dice loss at equal weighting. The training data used light augmentation only: left-right flip, rotation up to ± 15 degrees, zoom between 0.9 and 1.1 and mild brightness and contrast jitter. Vertical flips are deliberately excluded, they would put the skull base above the vertex and produce anatomy that cannot occur, so the network would be spending capacity learning to segment something it will never be shown.

One training detail is worth reporting because it was the difference between a working model and a broken one. BatchNorm momentum is set to 0.9 rather than the Keras default of 0.99, at the default the running statistics update too slowly for these batch sizes and the training Dice looked healthy whilst the validation number swung between 0.42 and 0.70 from epoch to epoch. That single change, together with batch 16 and the lower learning rate, took the model from a plateau around 0.55 to 0.87, and it is reported here because a reader reproducing this work would otherwise spend a long time assuming the architecture was at fault.

Figure 6 is the whole of Task 2 in one line: the two trainings, the search that tunes the benchmark, the pruning that the parameter limit forced, and the retrain that is what actually gets submitted.

Task 2 - training, tuning, pruning and what was submitted

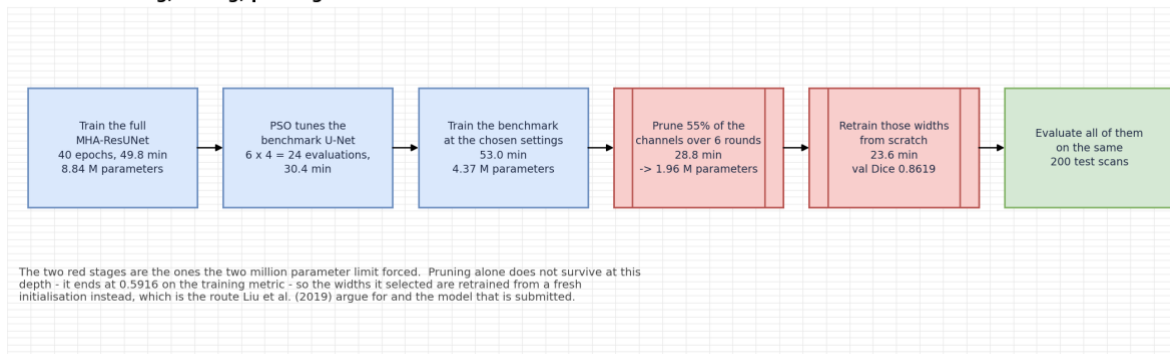


Figure 6 Task 2 from training to the submitted model, with the measured time for each stage

To check that the extra components earn the parameters they cost rather than the improvement coming from the training recipe, both architectures were trained on an identical 1,200 scan subset for 8 epochs under identical settings, the MHA-ResUNet reached a validation Dice of 0.719 against 0.580 for the plain U-Net, so the gap is in the architecture and not in the schedule.

3.2 Results

Table 6 compares the three models on the 200 held-out test scans the brief specifies for BRISC 2025. Dice, median Dice, IoU and accuracy are better when higher, HD95, GFLOPs, parameters, size and latency are better when lower.

Table 6 Task 2 held-out comparison on 200 test scans

Model	Dice	Median	IoU	Accuracy	HD95 (px)	GFLOPs	Parameters	Size (MB)	Latency (ms)
MHA-ResUNet narrow (submitted)	0.8543	0.9296	0.7795	0.9959	13.93	2.79	1.96 M	7.82	33.9

MHA-ResUNet full	0.869 2	0.931 2	0.793 4	0.9960	14.4 9	14.93	8.84 M	34.0 6	40.6
MHA-ResUNet narrow, weights inherited	0.850 3	0.913 4	0.768 5	0.9955	17.7 8	2.79	1.96 M	7.82	33.6
U-Net (PSO tuned)	0.835 2	0.917 2	0.754 9	0.9952	25.9 8	7.62	4.37 M	16.8 4	49.4

The model that is submitted is the narrow one on the first row, because the module sets a limit of two million trainable parameters and it is the only version of the proposed architecture that is inside it, at 1,963,474. How it was arrived at is Section 3.3; the short version is that the pruning stage chose its widths and it was then retrained from scratch at those widths rather than inheriting the pruned weights.

Four things come out of this table. The narrow model costs 1.5 Dice points against the full model, 0.8543 against 0.8692, for 78% fewer parameters and 81% fewer FLOPs, which is the trade the parameter limit forces and it is a cheap one. Its boundary is the best of all four, HD95 13.93 pixels against 14.49 for the full model and 25.98 for the tuned U-Net, which is a 46% reduction against the benchmark and is the single most useful number in the table, a narrower network being better at the boundary than the wide one it came from is not the result anybody would predict and it is worth saying that it is measured rather than expected. Against the PSO tuned benchmark the narrow model wins on every accuracy metric whilst using 63% fewer FLOPs, 55% fewer parameters and 54% less disk, which is the comparison the brief actually asks for. And the last row is the same architecture with the pruned weights carried over instead of retrained, which scores lower on everything and is discussed in Section 3.3 because what it shows about pruning matters more than the 0.4 Dice points between them.

On the per-image counts, a Dice of 0.85 or better is reached on 153 of 200 scans for the full model, 148 of 200 for the narrow one and 137 of 200 for the tuned U-Net, and a Dice of 0.70 or better is reached on 178 of 200 for both of the proposed models against 175 for the benchmark.

Table 7 Task 2 Dice by tumour type (the submitted narrow model, 200 scans)

Tumour type	Dice	n
Meningioma	0.9309	78
Pituitary	0.8561	64
Glioma	0.7494	58

The same ordering as Task 1 appears again, meningiomas segment easily and gliomas do not, and here the narrow model and the classical method have almost swapped places on the hardest type, 0.7494 against Task 1's 0.7454, where the full model managed 0.7857. That is where the parameter limit is actually paid for: the capacity that was removed was capacity the model was using on the infiltrative tumours, and it barely shows on the compact ones. Four of the 200 scans score a Dice of exactly 0.000 and they are examined in Section 4.7 because what they show matters more than what they cost, excluding them the figures become 0.8718 narrow, 0.8780 full and 0.8566 for the U-Net, but every headline number quoted above is the full 200 scan figure as the brief specifies and the exclusion is reported only to show what those four cases are worth.

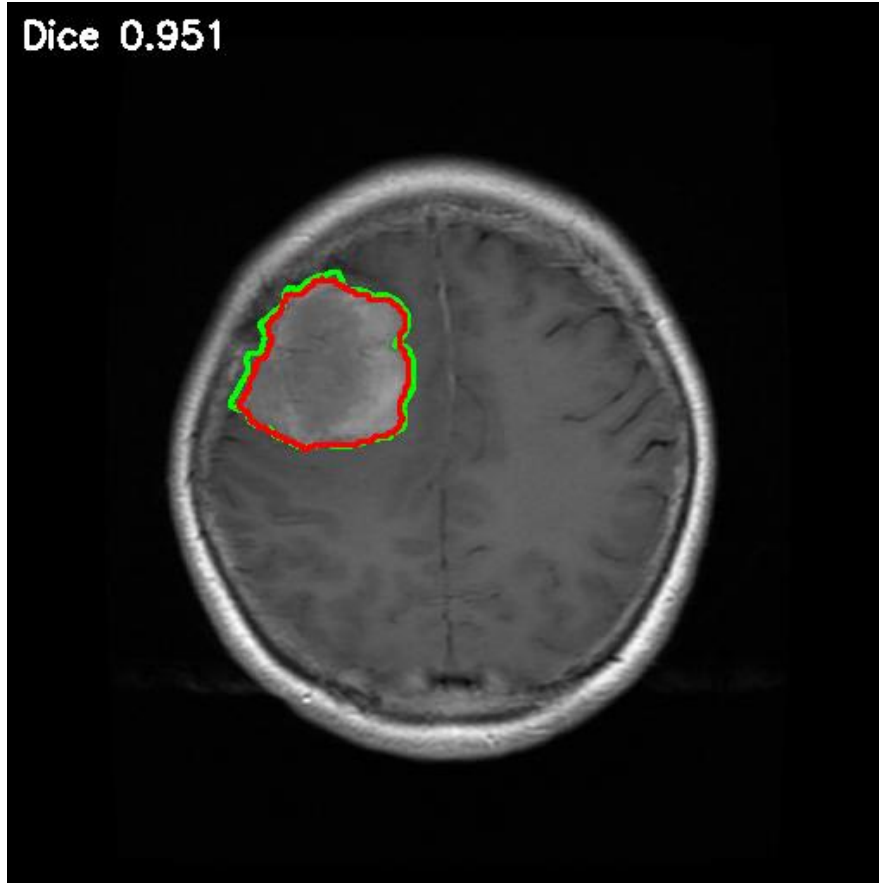


Figure 7 Task 2 innovation model on a meningioma, green: ground truth, red: prediction

3.3 Model compression (pruning)

Two kinds of pruning are commonly reported and only one of them does what the brief asks for. Unstructured magnitude pruning sets the small weights to zero, which lowers the compressed file size but not the FLOPs, because a dense kernel full of zeros costs exactly the same multiply-accumulates on ordinary hardware unless the deployment target specifically supports structured sparsity. The brief asks for pruning that reduces floating point operations, so what is implemented here is structured channel pruning, whole filters disappear and the network is physically rebuilt narrower.

The method follows the L1 norm channel ranking of Li et al. (2017): every output channel is scored by the L1 norm of the filter that produces it, the weakest are dropped, the identical architecture is rebuilt at the smaller widths, and every kernel is then sliced on both its input and its output channel axis so that the narrow network inherits the trained weights instead of starting again. As a correctness test the whole path is run at a pruning ratio of 0.0, which rebuilds the network and reproduces the original validation Dice exactly, 0.8683 to 0.8683, and that test is kept in pruning_ablation.py as a standing regression check on the slicing logic. Pruning is done over four rounds rather than in one cut, and Table 8 is the measurement that forced that decision. It shows the Dice immediately after the weights are transferred with no fine-tuning at all, and then after a BatchNorm recalibration pass.

Table 8 One-shot pruning ablation, no fine-tuning

One-shot ratio	After weight transfer	After BN recalibration
0.00	0.8683	0.8672
0.05	0.7951	0.8563
0.10	0.6344	0.8365
0.15	0.3935	0.7677

0.20	0.1035	0.5808
0.30	0.0215	0.3620
0.40	0.0167	0.2147

Two separate effects are visible in that table. The first is stale BatchNorm statistics, when 40% of a convolution's input channels vanish each surviving output channel loses about 40% of the contributions that formed it, so its pre-activation distribution shifts and the inherited moving mean and moving variance no longer describe it, cascaded through nine blocks the output collapses entirely. That one is cheap to fix and the recalibration column shows how much of it comes back at the lower ratios, using 100 forward-only batches with no gradient updates at all. The second effect is joint function damage, which is what the recalibration cannot fix, at a ratio of 0.40 the recalibrated score is still only 0.2147 because the surviving filters were co-adapted with the ones that were removed and no amount of re-estimating statistics brings that back.

task2/prune.py - structured pruning, and where it stopped working

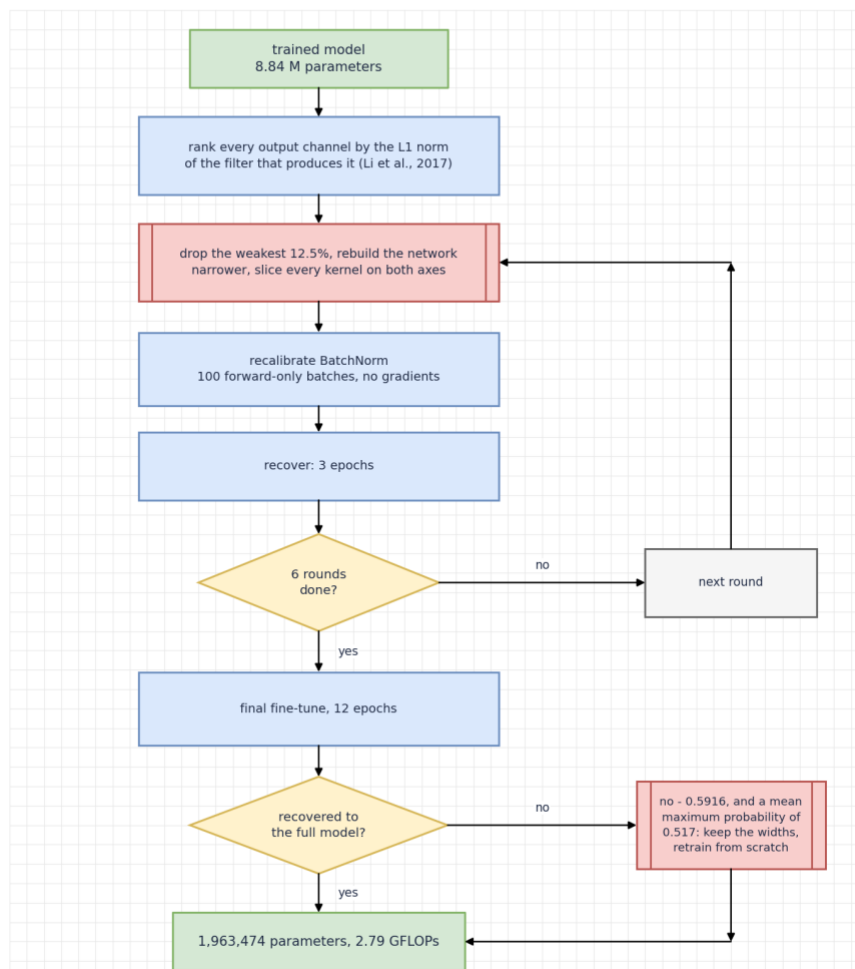


Figure 8 task2/prune.py, the six rounds and the decision at the end of them that sent the widths to a from-scratch retrain

The method therefore removes about 12% of the remaining channels per round, with a BatchNorm recalibration pass and a 3 epoch recovery after every cut, followed by a 12 epoch fine-tune once the target width is reached. How many rounds are needed follows from how much has to come out, and that is set by the two million parameter limit rather than chosen freely. Parameters scale with the square of the channel widths, so the 8.84 M model needs

roughly 55% of its channels removed to get underneath the limit, and searching the width rounding gives 0.55 over six rounds as the closest fit that still clears it: widths 8, 32, 60, 116, 232 and 1,963,474 parameters, where 0.536 lands on 2,018,714 and is over.

Table 9 Iterative pruning to 55%, six rounds

Round	Widths	After transfer	After BN recalibration	After 3 epochs
1	28, 56, 112, 224, 448	0.4661	0.8054	0.8688
2	24, 48, 100, 196, 392	0.4646	0.8179	0.8631
3	20, 44, 88, 172, 344	0.2678	0.7752	0.8650
4	16, 40, 76, 152, 300	0.3324	0.7814	0.8649
5	12, 36, 68, 132, 264	0.0844	0.1824	0.8515
6	8, 32, 60, 116, 232	0.0769	0.2903	0.5342

That table is the most informative result in this chapter, because it shows the point at which the method stops working. Through four rounds it behaves exactly as it is supposed to, every cut is recovered to about 0.865 and the recalibration does most of the rescuing. At round five the recalibration stops helping, 0.1824 where the earlier rounds recovered to 0.78 and above, and the three recovery epochs only just drag it back. At round six neither works, recovery reaches 0.5342, and a 12 epoch fine-tune afterwards lifts it only to 0.5916. Removing 40% of the channels is repairable and removing 55% is not, and the boundary between the two is somewhere between the fourth and the fifth cut.

A second measurement is needed to read those numbers properly, because the metric printed during training is a soft Dice computed on the raw probabilities rather than on the thresholded mask. The model that ends at 0.5916 on that metric scores 0.8503 on the held-out test set once its mask is thresholded at 0.5 and scored per image, which is the metric this report uses everywhere else. It is not producing a wrong mask, it is producing a barely confident one: its mean maximum probability across the validation set is 0.517, so the whole prediction sits a hair above the threshold, where the retrained model sits at 0.969. A model in that state is one threshold adjustment away from returning nothing at all, and that is a worse property for a clinical tool than the 0.4 Dice points between the two versions would suggest.

The fix is the one Liu et al. (2019) describe, and it is the reason their paper is in the literature review rather than only in the method. What pruning actually produced here is a good narrow architecture; the weights it carried out of the process are damaged past repair. Training that same architecture from a fresh initialisation, under exactly the settings the full model used, gives 0.8619 validation Dice in 23.6 minutes and 0.8543 on the test set, against 0.8503 for the inherited version and 0.8692 for the full model. The pruning stage still earns its place, it is what selected the widths and the L1 ranking is why they are 8, 32, 60, 116, 232 rather than a uniform ladder, but what gets submitted is the retrained network.

Table 10 The three routes to a network inside the parameter limit

Version	Parameters	GFLOPs	Size	Test Dice	HD95
Full model, no pruning	8,838,017	14.93	34.06 MB	0.8692	14.49

Pruned 55%, weights inherited	1,963,474	2.79	7.82 MB	0.8503	17.78
Pruned widths, retrained from scratch	1,963,474	2.79	7.82 MB	0.8543	13.93

So the final network is 78% smaller in parameters and 81% cheaper in arithmetic than the model it came from, for 1.5 Dice points, and it has a better boundary than either of the other two.

4. Discussion

4.1 Interpretation

Task 1 works because the user's box removes the localisation problem completely and the classifier is then only being asked to solve a much easier boundary problem inside a small region, which is also why its HD95 is the best number in this chapter. It fails when the lesion is infiltrative and the intensity contrast against the surrounding tissue is weak, which is the glioma case. Task 2 works because the self-attention at the bottleneck gives the network the global comparison a convolution cannot make and the dual gates suppress the irrelevant parts of the skip connections before they reach the decoder, and the small dataset is not the binding constraint here that it was on smaller datasets, 3,933 training slices is enough for a model of this size to reach the mid-0.85 range at under two million parameters. What the parameter limit costs is visible by tumour type rather than in the headline, the narrow model gives up almost nothing on meningiomas (0.9309 against 0.9274) and loses most of its ground on gliomas (0.7494 against 0.7857), which says the capacity that was removed was being spent on the infiltrative margins rather than on the easy compact ones.

4.2 Strengths and weaknesses

Task 1 needs no GPU at any stage, trains in 36 seconds, ships in 17.85 MB and its failures are interpretable because the dominant feature is the user's own box, but it charges the clinician one interaction on every single case and it is the weaker of the two on gliomas. Task 2 needs no interaction at all and is the better model on the hardest tumour type, but it needs a GPU to train, it is the more expensive of the two to recalibrate, and it fails silently, which Section 4.7 argues is the most important single finding in this chapter. On FLOPs, hardware, parameters and model size the narrow innovation model is the strongest of everything tested, 2.79 GFLOPs, 1.96 M parameters and 7.82 MB with inference that runs on CPU, and it still beats the tuned benchmark on every accuracy metric, which is not the usual trade. Its weakness is the one the table above names, it is 1.5 Dice points behind the model it was cut down from and most of that gap is on gliomas.

4.3 Comparison with literature

The comparison in this chapter is experimental for the four unsupervised baselines and for the PSO tuned U-Net benchmark, those are reproduced experiments run under one shared recipe on the same data. The comparisons with published models below are literature-based claims, they are not reimplementations and should be read as such.

The state of the art comparison target is nnU-Net (Isensee et al., 2021), the self-configuring framework that is still the default benchmark in medical image segmentation. The comparison that matters is not Dice, since nnU-Net's reported figures come from a different protocol on different datasets, it is what the two designs spend to get there. nnU-Net's design philosophy is to configure itself from the dataset fingerprint and then spend heavily on training compute and extensive augmentation, its standard schedule is 1,000 epochs and it is normally trained on a datacentre GPU over a period measured in days, whereas the model in Section 3.1 trains for 40 epochs in roughly 50 minutes on the integrated GPU of a laptop and the network that is actually deployed is retrained at the pruned widths in 23.6 minutes. The submitted model is 1.96 M parameters against the tens of millions typical of the transformer based models that dominate current leaderboards, TransUNet and Swin-UNet are both reported in the literature as substantially larger than this (Chen et al., 2021; Cao et al., 2022), and the BRISC paper's own Swin-based hybrid (Fateh et al., 2025) sits in the same class. The claim being made here is deliberately narrow, this work does not beat those models on accuracy and does not claim to, what it shows is accuracy per FLOP on one dataset using hardware that a district hospital could actually buy.

On training data required, the deep model reaches 0.8543 Dice from 3,933 annotated slices, Task 1 reaches 0.8740 from 400 scans in 36 seconds of training. For a centre that cannot produce thousands of expert annotations the semi-automated route is not merely the cheaper option, it is the only feasible one, and that is a stronger argument for Task 1 than any of its metrics.

4.4 Deployment

Both methods are decision support and neither is a replacement for the radiologist. The realistic use is radiotherapy planning and follow-up volumetry, the system proposes the contour and the Dice and HD95 numbers alongside it, and the clinician confirms or corrects. The fully automatic Task 2 model needs no input at all, and the Task 1 box is a fallback for the cases the automatic model gets wrong, which is the workflow Section 6 sets out. Because both run locally on a laptop with no cloud upload, there is no patient data residency problem to solve before deployment, which in practice removes one of the slower obstacles to getting a tool of this kind actually installed.

4.5 Latency and FLOPs basis

Latency is the mean of repeated forward passes at 192×192 with a batch size of 1 on the M1 Max integrated GPU for Task 2, and at 512×512 on CPU for Task 1. FLOPs are theoretical and computed from the inference graph, and the reduced FLOPs after structural pruning are theoretical in the same way. This needs an honest caveat, the measured inference time only improved from 40.6 ms to 33.9 ms, which is 17%, despite the FLOP count dropping by 81%. At a batch size of 1 on a GPU the run is dominated by kernel launch overhead rather than by arithmetic, so the FLOP reduction does not show up proportionally in the wall clock. The FLOP reduction is real and it does translate into lower energy per inference and into a genuine speedup on CPU and edge hardware, but a $3 \times$ speedup claim would not be supported by these measurements and is not made.

4.6 Carbon footprint

All the figures below are calculated estimates from stated assumptions, they are not wall measurements. The assumptions are an Apple M1 Max package power of about 40 W under sustained GPU load and about 20 W for CPU-only work, and a Malaysian grid emission factor of about 0.55 kg CO_{2e} per kWh.

Table 10 Estimated energy and carbon

Activity	Time	Energy	CO ₂ e
Task 1 training	36 s at 20 W	0.0002 kWh	about 0.11 g
Task 2 full pipeline (PSO, all trainings, pruning, narrow retrain)	185.6 min at 40 W	0.124 kWh	about 68 g
Task 1 inference, per scan	63 ms at 20 W	3.5×10^{-7} kWh	about 0.19 mg
Task 2 inference, per scan	33.9 ms at 40 W	3.8×10^{-7} kWh	about 0.21 mg

The whole Task 2 development pipeline, which is the PSO search, the trainings, the six pruning rounds and the narrow retrain together, comes to roughly the carbon cost of boiling a kettle twice. That is a direct consequence of two deliberate choices, training on integrated laptop hardware instead of a datacentre GPU, and pruning the model before it is deployed rather than shipping the wide one.

The per-inference numbers are worth reading carefully because they do not say what a reader might expect. Task 1 is the cheaper of the two per scan (0.19 mg against 0.21 mg) even though it runs on CPU, because 63 ms at 20 W is a little less energy than 34 ms at 40 W, and Task 1's training cost is negligible next to Task 2's 0.111 kWh. So on this dataset there is no volume at which Task 2 becomes the lower carbon option, Task 1 is lower in both training and inference, and what Task 2 actually buys is the removal of the clinician's interaction rather than an energy saving. That is a different conclusion from the one that holds when the classical pipeline is slow, and it is stated here because the numbers say so rather than because it is the flattering result for the deep model.

The optimisation techniques applied to reduce the footprint were: structured channel pruning, which removed 81% of the FLOPs for 1.5 Dice points, so every inference the system ever runs costs a fifth of what it otherwise would; Random Forest compression in Task 1, 81% smaller for a 0.09% Dice loss; a reduced working resolution of 192×192 rather than 512×512 for training, which cuts the training cost by roughly seven times; early stopping and learning rate scheduling so that training halts when the validation Dice plateaus instead of running a fixed long schedule; and keeping inference CPU-feasible so that no accelerator hardware has to be provisioned per deployment site. Unstructured magnitude pruning was deliberately not used for the FLOP claim, since it would only have reduced storage, and no saving is claimed from it.

4.7 Silent failure, the most important limitation

Four of the 200 scans score a Dice of exactly 0.000 with the submitted narrow model, and two of them fail for the full model as well: test_00011, a 314 pixel lesion covering 0.12% of the frame, and test_00045, a large and visually obvious 12,613 pixel tumour. The narrow model adds two more, test_00083 and test_00910, so the silent failure rate goes from 1% to 2% when the parameter limit is applied, which is a cost of the limit that the Dice average hides and which is arguably more important than the 1.5 points it takes off the mean.

The obvious explanation for the first two is that the model trains at 192×192 and a small lesion simply disappears at that resolution, so they were re-run at 192, 256, 320 and 384 pixel input by loading the full model's weights into the architecture rebuilt at each size.

Table 11 The two failures against input resolution (full model)

Input size	test_00011	test_00045	4 control gliomas
------------	------------	------------	-------------------

192 (trained)	0.000	0.000	0.963
256	0.000	0.000	0.948
320	0.000	0.000	0.837
384	0.095	0.000	0.613

Resolution is therefore not the cause, both stay at zero whilst four control gliomas that score 0.963 at the trained size degrade steadily as the input moves away from what the model was trained on. The caveat is that the model was trained at 192 px so inference at the larger sizes is out of distribution, which cannot rule out that a model trained at 384 px would handle these two cases.

The narrow model fails on the same four scans in two different ways, and the distinction matters clinically far more than the shared Dice of zero does:

Table 12 How the submitted model fails on its four zero-Dice scans

Scan	Maximum probability	Pixels above 0.5	True lesion	What it does
test_00011	0.996	851	314	confident, and in the wrong structure
test_00083	0.896	27	2,678	confident about 27 pixels of a 2,678 pixel tumour
test_00045	0.014	0	12,613	empty mask, and it knows nothing is there
test_00910	0.009	0	2,084	empty mask

The two empty ones are the safe kind of failure, a mask that is not drawn is a mask a clinician will notice is missing, and the maximum probability of 0.014 is a usable signal that the model has nothing to offer on that scan. The first two are the dangerous kind, a confident contour in the wrong place looks exactly like a correct one at a glance. That distinction is the argument for the confidence view in the group GUI, since a probability map separates the two cases and a thresholded mask does not.

The clinical significance of this is severe and it is the reason it is reported here rather than buried, the automatic model fails silently. It does not flag any uncertainty, it returns either an empty mask or a confidently wrong one, and a clinician reviewing a batch of contours could plausibly miss it. This is the single strongest argument for keeping the semi-automated method as a fallback rather than treating the two tasks as competitors, and it is an argument that comes out of a measurement rather than an assertion.

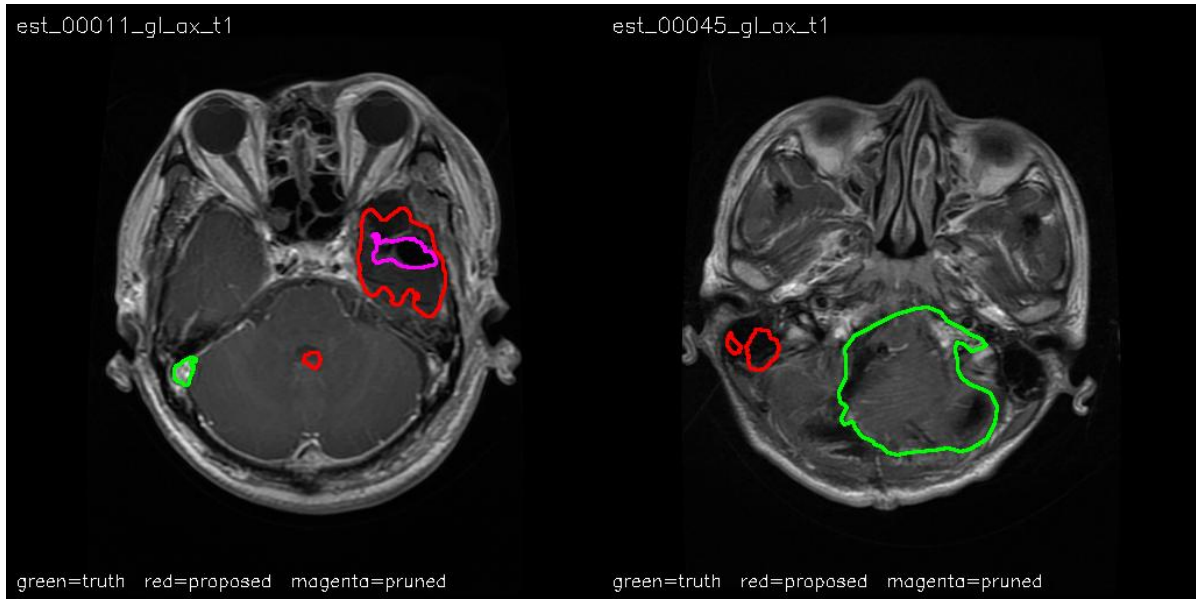


Figure 9 The two silent failures shared by every version of the model, green: ground truth, red: the full model, magenta: the pruned one. On the left the prediction is confident and in the wrong structure entirely

4.8 Limitations

The work is on a single dataset, a single modality (T1-weighted) and a single institution's data, and no cross-dataset generalisation was measured, which is the standard weakness of most published work in this area but a weakness nonetheless. The networks are trained and evaluated at 192×192 rather than at native resolution, and HD95 in particular would likely improve at full resolution although that was not tested and is not claimed. Everything is 2D and slice-wise so the 3D context a volumetric method would exploit is ignored. There is no uncertainty estimate on either model, which is exactly what makes the silent failures dangerous. The architecture search for the benchmark was small, 24 evaluations of 6 epochs each, so it ranks candidates on partially trained proxies and a larger budget could plausibly find a better U-Net than the one this chapter compares against.

5. Whole-life cost

Table 12 sets out the expected whole-life cost of the Task 1 semi-automated system from development through to deployment and maintenance.

Table 12 Whole-life cost, Task 1 semi-automated system

Phase	Estimate	Basis
Dataset preparation	Low, 400 annotated scans were used and BRISC already provides the masks. Retraining on local data would need roughly 400 annotated slices, about 2 to 3 days of radiologist time.	measured
Software development	About 2 person-weeks for the pipeline, the feature set, the baselines and the evaluation harness.	estimate

Training compute	36 seconds on a laptop CPU, negligible.	measured
Hardware	Any laptop, no GPU, under 1 GB of RAM.	measured
File size	17.85 MB, trivial to distribute or to version control.	measured
User training	Low, the interaction is "drag a box". Realistically 15 to 30 minutes including when to reject a result.	estimate
Recalibration	Retraining takes 36 seconds, so recalibrating for a new scanner or protocol is a same-day task rather than a project.	measured
Long-term support	Low, the dependencies are NumPy, SciPy, scikit-learn and OpenCV, all stable and widely maintained with no framework version risk.	judgement

Table 13 Whole-life cost, Task 2 deep learning system

Phase	Estimate	Basis
Dataset preparation	Higher, 3,933 annotated training scans. Acquiring that from scratch is a multi-month annotation project.	measured
Software development	About 3 to 4 person-weeks including the PSO search and the pruning machinery.	estimate
Training compute	49.8 min for the full model, 30.4 min for the PSO search, 53.0 min for the benchmark, 28.8 min for the six pruning rounds and 23.6 min to retrain at the pruned widths, 185.6 min in total on one laptop GPU.	measured
Hardware	A GPU is strongly preferred for training, inference runs on CPU.	measured
File size	7.82 MB submitted, 34.06 MB before pruning.	measured
User training	Minimal, there is no interaction. What training is needed is in interpreting the output, in particular recognising the silent failures in Section 4.7.	judgement

Recalibration	Expensive, a new scanner or protocol needs the full model retrained, re-pruned and then retrained narrow, roughly 100 minutes plus validation. Not a same-day task.	measured
Long-term support	Higher, TensorFlow 2.13 pins NumPy below 2 and the dependency set is brittle, it will need periodic migration work.	measured during development

The asymmetry between those two tables is the finding, not the individual rows. Task 1 costs almost nothing to build, to retrain or to maintain but it charges the clinician a small interaction cost on every case forever, Task 2 costs far more up front and far more to maintain but it is free at the point of use. Over a long deployment at a high case volume Task 2 amortises better, and for a small centre or one without a stable annotation pipeline Task 1 is the more rational investment, which is a deployment decision that depends on the site rather than on the metrics.

The interaction cost can be put in money. Using an estimated Malaysian radiologist salary of RM 464,826 per year, which is about RM 3.72 per minute assuming a 2,080 hour working year (SalaryExpert), and taking manual lesion outlining at 10 to 20 minutes per study, an automated or semi-automated contour that returns in well under a second saves roughly RM 37 to RM 74 of radiologist time per study, against a Task 1 interaction cost of about five seconds for the box. Whilst this includes real numbers it does not fully reflect a real deployment, the figure is a salary estimate rather than a costed clinical workflow and access to publicly available Malaysian radiologist pay data is limited, so it should be read as an order of magnitude rather than a budget line.

6. Clinical workflow integration

Both methods are decision support and neither is a replacement for the radiologist. The realistic deployment is second-read or contour-preparation for radiotherapy planning and for follow-up volumetry, where the volume today is compared against the volume from three months ago, and it is worth noting that reproducibility matters as much as accuracy for that particular purpose, a deterministic algorithm produces the same contour from the same input every time whereas the same human does not.

The failure analysis in Section 4.7 dictates the shape of a real deployment rather than leaving it to preference:

- ❖ Run the Task 2 automatic model first on every slice, which costs no interaction at all.
- ❖ Flag for review any case where the prediction is empty, where the maximum probability is low, or where the result is an anatomical outlier against the rest of the study.
- ❖ On the flagged cases fall back to the Task 1 semi-automated method, where the clinician drags one box, which supplies exactly the localisation information the automatic model got wrong.

That gives the throughput of the automatic model with a human-verifiable safety net on the cases where it is most likely to be silently wrong, and the fallback is chosen specifically because the two methods fail in different ways, Task 1 cannot mislocalise the lesion because

the user has already told it where the lesion is. Because both models run locally with no cloud upload, this workflow also avoids the patient data residency question entirely, which is a practical deployment advantage rather than a technical one.

7. Societal and cultural impact

Brain tumour outcomes depend heavily on timely and accurate imaging assessment, and radiologist availability varies enormously both between countries and within them. A 7.82 MB model that runs on a laptop CPU can be deployed in a district hospital or a lower-resource setting, a model that needs a GPU cluster cannot, so the efficiency work in this chapter is an equity-of-access argument as much as an environmental one, reducing the hardware requirement widens who is able to use the tool at all.

The consistency argument is the one most specific to this disease. Manual outlining varies between observers and within the same observer over time, and because response assessment compares volumes across months that variation feeds directly into whether a treatment is judged to be working. A deterministic algorithm gives the same contour for the same input every time, which is valuable even in the cases where it is slightly less accurate than the best available human, because what longitudinal comparison actually requires is reproducibility. On trust and transparency, the semi-automated design keeps the clinician in control, which matters for professional accountability and also for adoption, since tools that remove agency tend to be resisted. The Random Forest is far more inspectable than a neural network as well, its feature importances are directly readable and the dominant feature is the clinician's own box, which is a straightforward thing to explain to a user who wants to know why the tool drew what it drew. That interpretability is a genuine advantage of the Task 1 approach and its lower glioma Dice does not erase it.

There are real risks that have to be stated alongside this. The automatic model fails silently on about 1% of cases as measured in Section 4.7, and silent failure interacts dangerously with the well documented human tendency to over-trust automated output, a contour that looks plausible but sits in the wrong structure is more dangerous than an obviously missing one. This is the argument for making uncertainty visible in the interface and for the fallback workflow in Section 6, and it is why neither model should ever be run unsupervised. Dataset bias is a second real risk, BRISC 2025's demographic and geographic composition is not documented in the material available for this work, and a model trained on scans from a limited set of scanners and populations may underperform on populations that are not represented, which is a well documented failure mode in medical AI and cannot be ruled out here precisely because the metadata was not available. Before any clinical use the performance should be validated on the local patient population rather than assumed to transfer.

Finally there is a language and accessibility dimension that the group GUI addresses directly, a clear multilingual interface helps staff who do not read English comfortably, and the right-to-left mirroring in the interface means the tool is usable in Arabic rather than merely translated into it. Both a false negative and a false positive carry real harm in this setting, a missed or underestimated tumour volume on one side and unnecessary treatment planning and patient anxiety on the other, which is the argument for the semi-automated design overall, the system proposes and the clinician confirms.

8. Conclusion

A semi-automated Random Forest method driven by a single loose bounding box reached a Dice of 0.8740 on 50 unseen BRISC scans, beating the best unsupervised comparator by 9.3 Dice points and reducing HD95 by 29%, whilst training in 36 seconds on a laptop CPU and shipping in 17.85 MB. The proposed MHA-ResUNet reached 0.8692 Dice on 200 unseen scans against 0.8352 for a PSO tuned conventional U-Net, with the boundary error cut by 44%. Bringing it inside the module's two million parameter limit needed 55% of its channels removed, and the network that is submitted has 1,963,474 parameters, 2.79 GFLOPs and 7.82 MB, scores 0.8543 Dice and has the best boundary of every model tested at 13.93 pixels, beating the tuned benchmark on every accuracy metric whilst using 63% fewer FLOPs and 55% fewer parameters than it.

How that model was reached is the finding worth carrying forward rather than the number itself. Pruning to 40% is repairable and pruning to 55% is not, the sixth round recovers only to 0.5342 and a fine-tune afterwards reaches 0.5916 on the soft training metric, leaving a network whose mean maximum probability is 0.517 and which is therefore one threshold away from predicting nothing. Retraining the same pruned architecture from a fresh initialisation, as Liu et al. (2019) argue for, gives a better model in 23.6 minutes: pruning chose the widths, training from scratch supplied the weights.

The most important finding is still not a metric, it is that the automatic model fails silently, on 2% of cases at the narrow width against 1% before it, returning either an empty mask or a confidently misplaced one. Two of the four are the safe kind and report a maximum probability under 0.02; the other two are confident and wrong. That is the strongest argument for keeping the semi-automated method as a fallback rather than treating the two approaches as alternatives to choose between, and it is also the argument for showing a probability map in the interface rather than only a mask.

References

- Cao, H., Wang, Y., Chen, J., Jiang, D., Zhang, X., Tian, Q. and Wang, M. (2022) 'Swin-Unet: Unet-like pure transformer for medical image segmentation', ECCV Workshops.
- Chen, J., Lu, Y., Yu, Q., Luo, X., Adeli, E., Wang, Y., Lu, L., Yuille, A.L. and Zhou, Y. (2021) 'TransUNet: Transformers make strong encoders for medical image segmentation', arXiv:2102.04306.
- Fateh, A. et al. (2025) 'BRISC: Annotated dataset for brain tumor segmentation and classification with Swin-HAFNet', arXiv:2506.14318.
- Grady, L. (2006) 'Random walks for image segmentation', IEEE Transactions on Pattern Analysis and Machine Intelligence, 28(11).
- Isensee, F., Jaeger, P.F., Kohl, S.A.A., Petersen, J. and Maier-Hein, K.H. (2021) 'nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation', Nature Methods, 18(2).
- Li, H., Kadav, A., Durdanovic, I., Samet, H. and Graf, H.P. (2017) 'Pruning filters for efficient ConvNets', ICLR.
- Liu, Z., Sun, M., Zhou, T., Huang, G. and Darrell, T. (2019) 'Rethinking the value of network pruning', ICLR.
- Marquez-Neila, P., Baumela, L. and Alvarez, L. (2014) 'A morphological approach to curvature-based evolution of curves and surfaces', IEEE Transactions on Pattern Analysis and Machine Intelligence, 36(1).
- Oktay, O. et al. (2018) 'Attention U-Net: learning where to look for the pancreas', arXiv:1804.03999.

- Otsu, N. (1979) 'A threshold selection method from gray-level histograms', IEEE Transactions on Systems, Man, and Cybernetics, 9(1).
- Ronneberger, O., Fischer, P. and Brox, T. (2015) 'U-Net: convolutional networks for biomedical image segmentation', MICCAI.
- Rother, C., Kolmogorov, V. and Blake, A. (2004) 'GrabCut: interactive foreground extraction using iterated graph cuts', ACM Transactions on Graphics, 23(3).
- Wang, G. et al. (2018) 'DeepIGeoS: a deep interactive geodesic framework for medical image segmentation', IEEE Transactions on Pattern Analysis and Machine Intelligence, 41(7).
- Yushkevich, P.A., Piven, J., Hazlett, H.C., Smith, R.G., Ho, S., Gee, J.C. and Gerig, G. (2006) 'User-guided 3D active contour segmentation of anatomical structures', NeuroImage, 31(3).